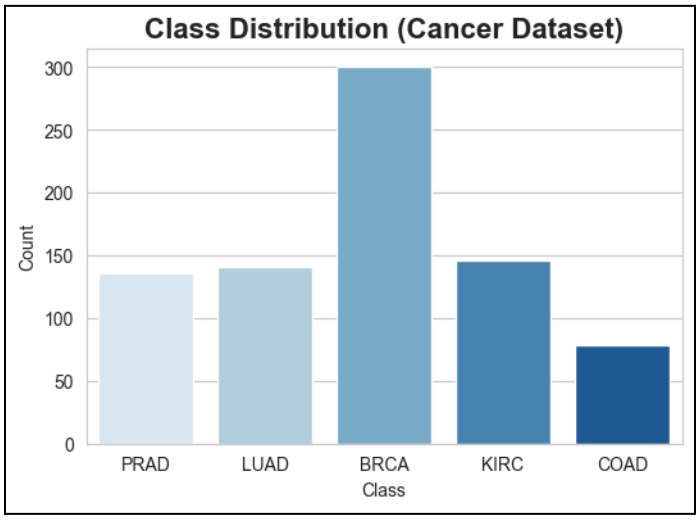
Ashley Woertz
CSCI1851
Professor Singh
2 March 2026

Homework 2 Final Reflection

Cancer Genomics Dataset:

The cancer genomics set contains 5,479 numerical features, each representing the RNA-Seq gene expression level of a specific gene (named `gene_XX`) These expression levels were measured using illumina HiSeq platform across tumor samples (Source: UCI). The target variable has multiple classes which are cancer types: BRCA, KIRC, LUAD, PRAD, and COAD. Since the dataset was pre-feature-selected and cleaned, I did not remove any redundant/irrelevant genes. The only preprocessing is optional feature scaling using a `StandardScaler`. There are 801 total samples; there are many more features than samples meaning high dimensionality. There are no missing values. The classes are somewhat unbalanced, with counts: BRCA: 300, KIRC: 146, LUAD: 141, PRAD: 136, and COAD: 78. Therefore, I used stratified splitting for train-test splitting.



Model Performance:

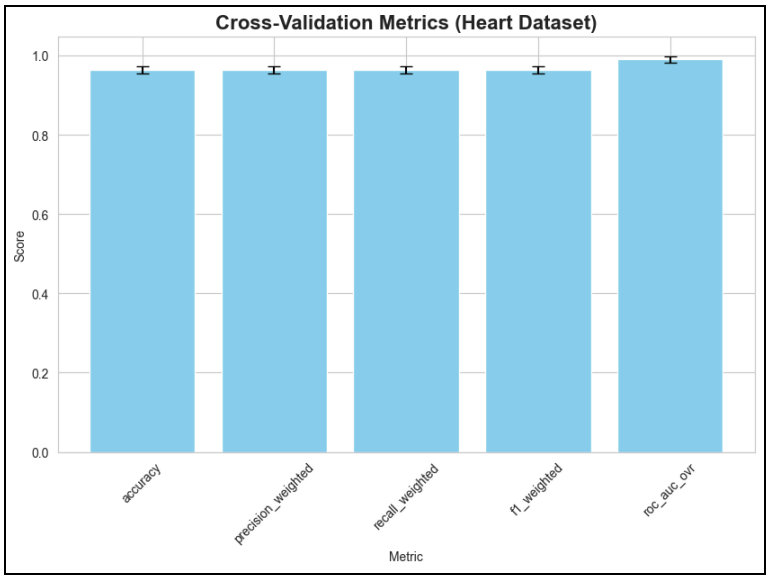
On default models (with no hyperparameter tuning) there was the following model performance:

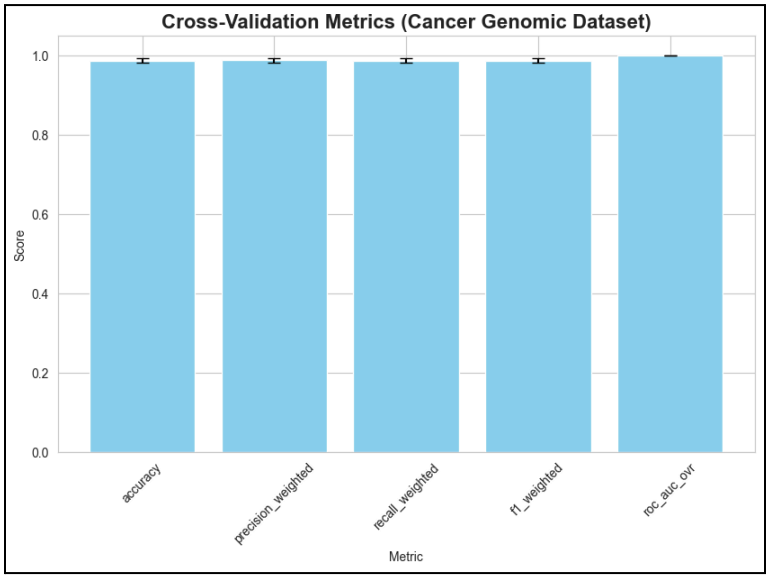
Dataset	Accuracy	Precision	Recall	F1	ROC-AUC
Heart Disease	0.9513	0.9467	0.9530	0.9498	0.9890
Cancer Genomics	0.9876	0.9880	0.9876	0.9875	0.9937

I thought both datasets had very strong performance especially without hyperparameter tuning. The model for the Cancer Genomic dataset outperformed the Heart Disease model for all metrics. The ROC-AUC scores were especially high. I saw an EdStem post about removing duplicates - I found that the Cancer Genomic set had no duplicates and was still getting 0.9937.

As for the cross-validation, here were the results:

Dataset	Accuracy	Precision (weighted)	Recall (weighted)	F1 (weighted)	ROC-AUC (one-vs-rest)
Heart Disease	0.9639 ± 0.0085	0.9643 ± 0.0086	0.9639 ± 0.0085	0.9639 ± 0.0085	0.9908 ± 0.0081
Cancer Genomics	0.9888 ± 0.0061	0.9892 ± 0.0058	0.9888 ± 0.0061	0.9887 ± 0.0061	0.9998 ± 0.0003



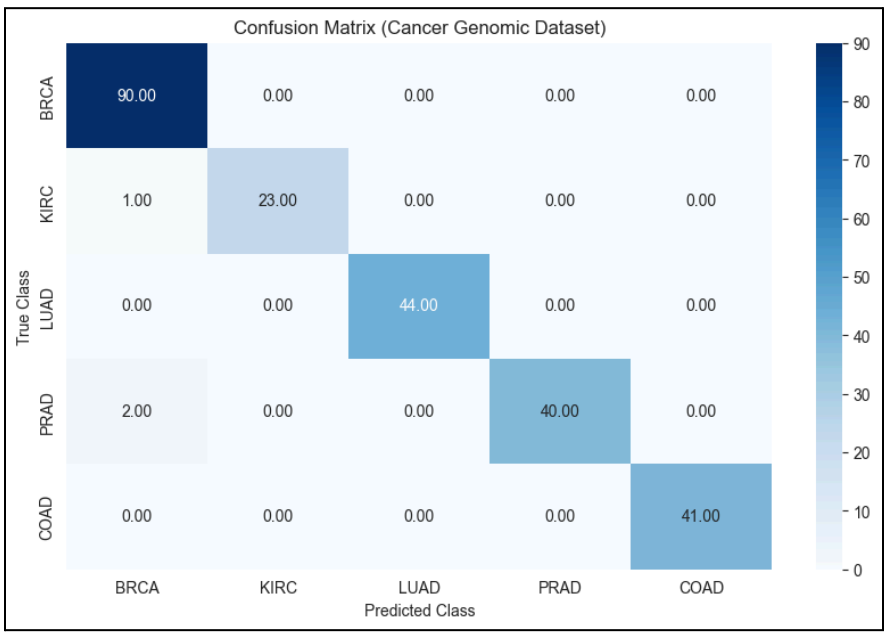


The cross-validation results are consistent with the default model performance so it seems that neither is overfitting. The Heart Disease model has slightly more variance but given the fewer samples and features this makes sense. The ROC-AUC score for the Cancer Genomics model is really close to 1 suggesting the model accurately distinguishes between classes.

The Cancer Genomics dataset was easier to classify than the Heart Disease dataset. I would have thought that the dataset with fewer features (Heart Disease, with 13) would be easier to classify but the RNA-Seq expression seems to be well correlated within distinct clusters by cancer type. Another side to 'easier' though

is computation. The Cancer Genomics dataset originally took me 120+ minutes to run, once I adjusted `n_jobs`, I was able to decrease the time to about 17 minutes per cell. The high dimensionality of the Cancer Genomics dataset made it harder in this regard.

For Heart Disease, the errors were even across the target variable as the precision was 0.9467 and recall 0.9530 so there's only slightly more false negatives than false positives. For the Cancer Genomics dataset, the precision and recall were nearly exactly the same, but this is more complicated as this is a multiclass classification problem. Using a confusion matrix, I was able to see that the only misclassifications were 1 KIRC sample classified as BRCA and 2 PRAD samples predicted as BRCA. These errors were around BRCA probably because it is the dominant class.



Hyperparameter Tuning:

I explored three hyperparameters with grid search: `learning_rate` with ranges from 0.1 → 1, `max_depth` with ranges from 2 → 4, and `n_estimators` from range 10 → 100. Mean and standard deviation of cross-validation accuracy

was used to score the parameters, which is done automatically by `best_score_` for `GridSearchCV`.

For the Heart Disease dataset, the most optimal hyperparameters were `{'learning_rate': 1, 'max_depth': 4, 'n_estimators': 50}` resulting in a mean CV score of 0.9916173786053304. I think those worked well because there are only 13 features so larger updates would benefit the model. Also, because `n_estimators = 50` was the best, it seems like there isn't much benefit to more trees. Here is the head of the DataFrame I printed during tuning:

Learning Rate	Max Depth	Number of Estimators	Mean CV Score	Std CV Score
1.0	4	50	0.991617	0.008964
1.0	4	10	0.990574	0.008596
1.0	4	100	0.987946	0.008901
1.0	3	100	0.986948	0.009922
0.1	4	50	0.986547	0.008894

For the Cancer Genomics dataset, the most optimal hyperparameters were `{'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 50}` resulting in a mean CV score of 0.9998511123201976. I think this was the best combination because the small learning rate prevents early overfitting when there are so many features. The lower max depth might show that the target variable was linearly separable. Here is the head of the DataFrame for the best hyperparameters during tuning:

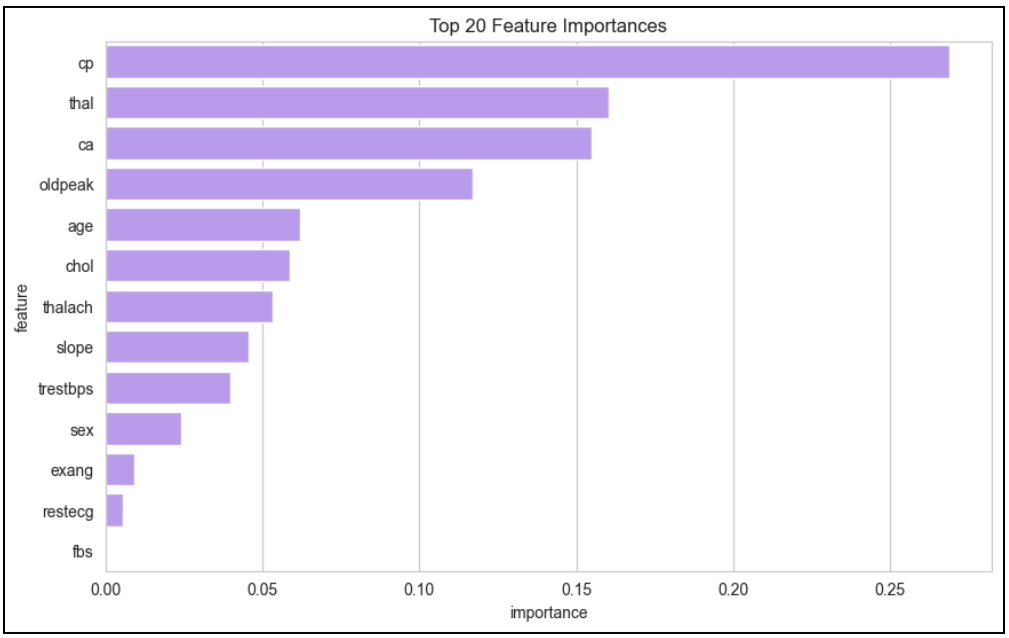
Learning Rate	Max Depth	Number of Estimators	Mean CV Score	Std CV Score
0.1	3	50	0.999851	0.000074
0.1	3	100	0.999847	0.000107
0.1	2	100	0.999837	0.000145

0.1	2	10	0.999825	0.000084
0.1	2	50	0.999824	0.000135

Feature Importance and Interpretation:

<u>Heart Disease dataset</u>:

The most predictive features were `cp` (chest pain type, 0.269), `thal` (thalassemia, 0.160), and `ca` (number of major vessels, 0.155) which account for 58% of importance. Logically, this makes sense these seem like appropriate risk factors for heart disease.

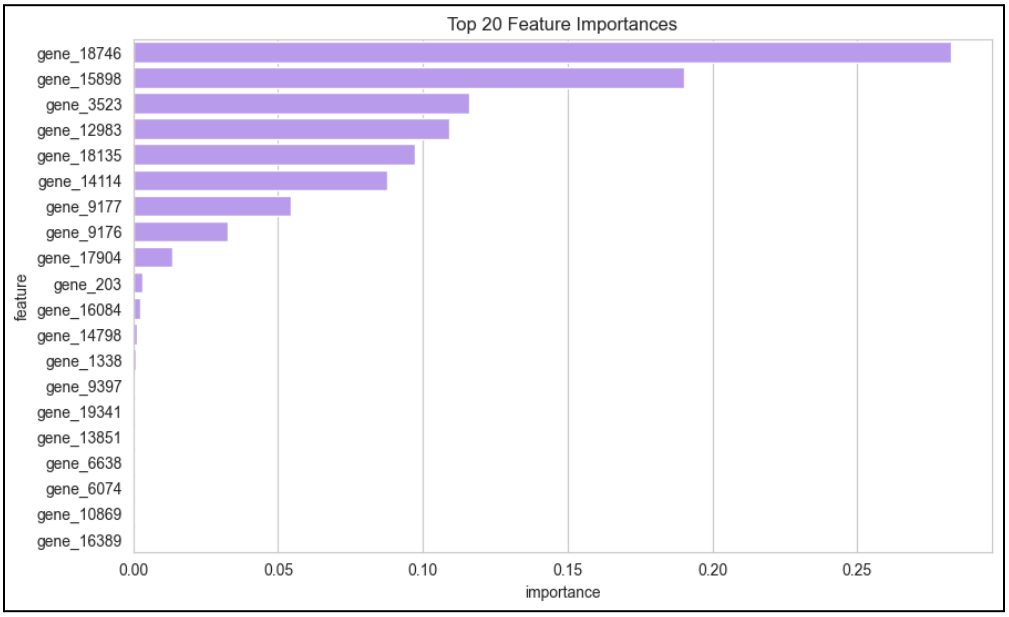


Feature	Importance
`cp`	0.268701
`thal`	0.160464
`ca`	0.155068

`oldpeak`	0.117060
`age`	0.062003
`chol`	0.058889
`thalach`	0.053325
`slope`	0.045564
`trestbps`	0.039771
`sex`	0.024209

<u>Cancer Genomics dataset</u>:

Despite there being over 5,000 features, the importance was concentrated around just a few genes. `gene_18746` accounts for 28.7% of the importance, then `gene_15898` (19.0%), and `gene_3523` (11.6%). Cancer type seems to be driven by just a few expression differences.



Feature	**Importance**

`gene_18746`	0.28230
`gene_15898`	0.19030
`gene_3523`	0.11609
`gene_12983`	0.10924
`gene_18135`	0.09738
`gene_14114`	0.08785
`gene_9177`	0.05428
`gene_9176`	0.03268
`gene_17904`	0.01342
`gene_203`	0.00331

<u>HW1 Lasso Model on Cancer Genomics dataset</u>:

Without changing the model architecture for HW1, I could not run the `get_feature_importance()` method on the Lasso model. However, I was able to get important features by sorting by the highest model coefficients. The most important features were `gene_17801`, `gene_11903`, and `gene_11576` however the coefficients were really quite low. There was no one feature that was super important like in the other models. Lasso seems to have low feature importances and uses all of them just a little bit.

Feature	Coefficient
`gene_17801`	0.051929
`gene_11903`	0.034760

`gene_11576`	0.031309
`gene_5578`	0.025226
`gene_3448`	0.025109
`gene_8032`	0.021855
`gene_1096`	0.020569
`gene_16372`	0.019585
`gene_6748`	0.019036
`gene_6976`	0.018024

Comparison to HW1:

Model	**MSE**	**RMSE**	**R^2**	**MAE**
Lasso - Full Feature Set	0.0475	0.2180	0.9798	0.1534
Lasso - Top 100 Lasso Features	2.0357	1.4268	0.1343	1.2912
Lasso - Top 100 GB Features	1.6366	1.2793	0.3040	1.1477

Training the HW1 Lasso model (`GenomicAgeRegressor`) on the full feature set resulted in strong performance, an R^2 of 0.980 and RMSE of 0.218. Restricting to the reduced top-K feature set where k = 100, resulted in a performance drop. The R^2 was now only 0.134 even though these features were considered the most important. Given there are over 5,000 features in this set, despite the high importance on some of the top few, the model no longer worked with the reduction. This is probably because Lasso distributes across many genes. The top 100 gradient boosting features were somewhat better, with an R^2 of 0.304. This was still much worse than the full-feature model, though. This is probably

because we are going from a classification to a regression task. In both cases the full feature set was clearly much superior for the Lasso task.

Discussion:

The models in this dataset may not be generalizable because the data they were trained on were artificial samples that were well-processed and cleaned. I think that real biological data samples like these will have a lot more variation and noise because of differences between humans. I don't know how these models would perform against raw data despite the extremely high accuracy. Gradient boosting doesn't assume a linear relationship or normal distribution of data.

The extremely high ROC-AUC score and accuracy do make me wonder if there is overfitting (98%+). The low standard deviations do show that the results are pretty stable especially across folds. For the boosted model though, the feature importance where one gene accounted for 28% suggested to me that the model may be overrelying on just a handful of expressions. For the heart disease dataset, the most optimal learning rate was 1.0 which also might cause the model to overfit as it takes large steps and maybe memorizes training data, however, the performance was good.

The datasets had very different optimal hyperparameters. This does show how there is sensitivity to hyperparameters but when I printed out the top 5 hyperparameter combinations for the gradient boosted model, they all had >0.9998 mean CV score which suggests there wasn't just *one* combination that would work but there is some leeway.

As for data quality, for the cancer genomics set, there was a sizable class imbalance with BRCA overrepresented while COAD underrepresented. This could lead to bias towards the majority class, and the only mistakes in the confusion matrix were falsely predicted to be BRCA, however the overall scores were still great. Again, the data being pre-cleaned and processed does make me wonder if the quality does not represent real-world data well.